



FOR SPRING DEVELOPERS MOVING TO QUARKUS

What Changes at Build Time

Spring Boot to Quarkus, the flagship

Quarkus did not ignore your setting. It had already used it, at build time.

One payment service written twice, in Spring Boot and in Quarkus. Controllers, injection and configuration translate almost line for line. Two things do not: who owns the transaction when you save, and which of your settings are decided before the application starts.

The comparison is against DEFAULT Spring on the normal JVM. With Spring AOT, conditions are evaluated at build time too.

Every version on this sheet was current on 23 September 2026 and will not stay current. The method is the durable part: run the script, read what came back, and write down the result rather than the expectation.

In scope

- The Spring annotations and their Quarkus counterparts
- Spring Data save() against Panache persist()
- A bean conditional on a property: decided at start, or at build
- What the build time mismatch check catches, and what it misses

Out of scope

- Startup time and memory. Nothing here measures them
- Native images, and dev mode
- Anything about how this channel is produced, which is not the subject

Every result on this page came from running the built jars, not from a test harness, except where a claim says it is asserted by the test suites.

01 The move, in one table

@RestController, @GetMapping	@Path, @GET: transfers
@Service, constructor injection	@ApplicationScoped, same constructor: transfers
@Value("\${...}")	@ConfigProperty(name = ...): transfers
save() is transactional by default	persist() needs @Transactional: rewired
@ConditionalOnProperty, read at start	@IfBuildProperty, decided at build: bites

The last row is the whole video. A value used at build time is not runtime configuration.



02 save() owned the transaction. persist() does not.

The Spring service has no @Transactional anywhere and still writes the row, because Spring Data's save() opens its own transaction.

Translate it line for line to Panache and persist() throws. It expects you to decide where the unit of work lives. Add @Transactional to the service method and the row is written. To be fair to Quarkus, this error names its own fix.

```
spring-payments: PaymentService.java
```

```
public Long record(long amountInMinorUnits, String currency) {  
    return repository.save(new Payment(amountInMinorUnits, currency)).getId();  
}
```

```
quarkus-payments: PaymentService.java
```

```
// The Spring service, translated line for line. No transaction anywhere.  
public Long recordLikeSpring(long amountInMinorUnits, String currency) {  
    Payment payment = new Payment(amountInMinorUnits, currency);  
    repository.persist(payment);  
    return payment.id;  
}  
  
// The fix: the unit of work is declared, because persist() does not own one.  
@Transactional  
public Long record(long amountInMinorUnits, String currency) {  
    Payment payment = new Payment(amountInMinorUnits, currency);  
    repository.persist(payment);  
    return payment.id;  
}
```

```
recordLikeSpring throws: "TransactionRequiredException: Transaction is not active, consider adding @Transactional to your  
method to automatically activate one."
```

Spring writes the row with no @Transactional; Panache persist() throws without one and writes with one.

Asserted by the test suites in the course repository, against Postgres.

03 True. Still off.

The same property gates the same FraudCheck bean in both apps. Each app is built once with the property false, then started with it set to true.

Spring reads the environment at startup, sees true, and builds the bean. Quarkus made the decision during the build: built with false, the bean was removed before the application ever ran, and starting it with true cannot bring back something that is no longer there.

```
● ● ● spring-payments: FraudCheck.java
```

```
@Component
@ConditionalOnProperty(name = "payments.fraud-check.enabled", havingValue = "true")
public class FraudCheck { ... }
```

```
● ● ● quarkus-payments: FraudCheck.java
```

```
@ApplicationScoped
@ifBuildProperty(name = "payments.fraud-check.enabled", stringValue = "true")
public class FraudCheck { ... }
```

```
● ● ● Built with false, started with true
```

```
Spring   GET /fraud-check {"fraudCheck":"present"}
Quarkus  GET /fraud-check {"fraudCheck":"absent"}
```

Spring reported present and Quarkus reported absent, from the built jars.

Run on a real machine, 2026-09-23. Reproduce it with `scripts/verify-hook.sh` in the course repository.

"Properties set at runtime have absolutely no effect on the bean resolution using `@IfBuildProperty` / `@UnlessBuildProperty`."

Quarkus CDI reference, quarkus.io/guides/cdi-reference, checked 2026-09-23

04 What the check catches, and what it misses

Quarkus does check for this kind of mistake in its OWN configuration. Change `quarkus.application.name` at runtime and it warns you at startup.

Set the check to fail and start the service with your property set to true: it starts normally, prints no warning, and the fraud check is still absent. The check protects Quarkus settings. It does not protect a property you used in `@IfBuildProperty`.

Quarkus's own build time setting

```
$ java -Dquarkus.application.name=renamed -jar quarkus-run.jar
WARN Build time property cannot be changed at runtime:
- quarkus.application.name is set to 'renamed' but it is build time
  fixed to 'quarkus-payments'.
```

Your property, with the check set to fail

```
$ java -Dquarkus.config.build-time-mismatch-at-runtime=fail \
-Dpayments.fraud-check.enabled=true -jar quarkus-run.jar
INFO quarkus-payments ... (powered by Quarkus 3.39.5) started
$ curl localhost:8082/fraud-check
{"fraudCheck":"absent"}
```

The warning appeared for `quarkus.application.name`; nothing appeared for `payments.fraud-check.enabled`, even with the check set to fail.

Run on a real machine, 2026-09-23. Reproduce it with `scripts/verify-hook.sh` in the course repository.

05 Spring does this too, if you ask

With Spring's ahead of time processing, the kind used for native images, conditions are evaluated at build time as well. So this is not Quarkus being strange and Spring being normal. It is a choice about when decisions are made: default Spring makes them at start up, Quarkus makes many of them at build time, all the time.

"Environment properties that impact the presence of a bean (`@Conditional`) are only considered at build time."

Spring Framework reference, ahead of time optimizations, checked 2026-09-23

Before you trust a Quarkus service in production

1. Every write path declares @Transactional where the unit of work lives
2. You know which of your properties are read at build time, and they are set in the build, not the deployment
3. No property used in @IfBuildProperty is expected to change per environment
4. quarkus.config.build-time-mismatch-at-runtime is set, knowing what it does not cover
5. scripts/verify-hook.sh passes on a fresh clone

Everything else

 Site https://code-with-sam-dev.github.io	 GitHub https://github.com/code-with-sam-dev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me sadza https://buymesadza.com/codewithsam
 X https://x.com/CodeWithSamDev	The course repository https://github.com/code-with-sam-dev/spring-to-quarkus
 TikTok https://www.tiktok.com/@codewithsamdev	Quarkus CDI reference https://quarkus.io/guides/cdi-reference
 Instagram https://www.instagram.com/codewithsamdev	Quarkus configuration reference https://quarkus.io/guides/config-reference
 Facebook https://www.facebook.com/codewithsamdev	Spring AOT reference https://docs.spring.io/spring-framework/reference/core/aot.html

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. PostgreSQL is a trademark of the PostgreSQL Community Association. Quarkus is the property of its respective owner. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-23 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.